

Testing Algorithms for Acoustic Species Identification and Using Manual Annotations of Sonotype for Species Identification

Summer Project Report
June - July 2025

Submitted by:
Arman Singh Jaryal
National Institute of Science Education and Research
(NISER), Bhubaneswar



Indian Institute of Science, Bangalore



Centre of Ecological sciences
Supervisor: Prof. Rohini Balakrishnan

July 2025

CERTIFICATE



This is to certify that Mr. **Arman Singh Jaryal** of the National Institute of Science Education and Research (NISER), Bhubaneswar has successfully completed his summer internship during the period of June-July 2025 on the topic of "**Testing Algorithms for Acoustic Species Identification and Using Manual Annotations of Sonotype for Species Identification**" at Animal Communication and Bioacoustics Lab ,Centre of Ecological Sciences.

Indian Institute of Science, Bangalore

Dr. Rohini Balakrishnan
Supervisor
Animal Communication and Bioacoustics Laboratory
Centre of Ecological Sciences
Indian Institute of Science, Bangalore

DECLARATION

I hereby declare that I am the sole author of this **Report** in partial fulfillment of the requirements for a **Summer Internship Project** from Indian Institute of Science (**IISC**), Bangalore.

Signature of the Student

Date:

The project work reported in the report entitled “**Testing Algorithms for Acoustic Species Identification and Using Manual Annotations of Sonotype for Species Identification**” was carried out under my supervision, in the Centre of Ecological Sciences at Indian Institute of Science (**IISC**), Bangalore, India.

Signature of the Guide/Supervisor

School:

Date:

ACKNOWLEDGMENTS

I want to express my heartfelt gratitude to Prof. Rohini Balakrishnan for her unwavering support and guidance. Her insightful feedback and mentorship have been instrumental in this research. I thank Mr. Arpit Omprakash the PhDs working on this project for their immense support, guidance, and encouragement.

I would like to thank the Cornell Lab of Ornithology and the Bioacoustics Research Program for providing access to Raven Pro, which was instrumental for the annotation and analysis of acoustic recordings in this project.

I would also like to thank PhDs working in the lab, for their exceptional supervision and mentorship. Their unwavering support, constructive criticism, and continuous motivation were instrumental in the completion of this report.

Abstract

Orthopteran species, such as crickets, exhibit rapid development, short generation times, and high reproductive output, and are particularly sensitive to changes in temperature and moisture. These characteristics make them ideal candidates for monitoring the ecological impacts of environmental fluctuations. Consequently, accurate identification of such acoustically active species is essential for effective biodiversity assessment, habitat monitoring, and conservation planning. Their sensitivity to microclimatic shifts enables them to function as early-warning bioindicators, reflecting the onset of environmental disturbances in real time.

Traditional mark and recapture can be invasive and can further threaten populations, particularly in sensitive or declining species. Passive acoustic monitoring (PAM) provides a non-invasive, scalable alternative that allows researchers to identify species based on their unique call signatures. At a multispecies level, PAM can reveal not only the presence of individual taxa but also the overall species richness of a given habitat. The implementation of PAM is accompanied by technical and biological challenges. Acoustic recordings often contain high levels of environmental noise and artifacts, while overlapping vocalizations within similar frequency ranges can hinder an accurate classification. Despite these constraints, PAM holds significant promise when paired with advanced signal processing and machine learning techniques and offers a powerful tool for ecological surveillance. In this study, I collected 41 acoustic recordings from individual calling crickets. Some recordings included overlapping calls from multiple species, resulting in 43 distinct annotated call events at the species level. Each call was manually analyzed using Raven Pro, where I extracted key acoustic parameters. In addition, I noted qualitative diagnostic traits including the number of syllables per call and the presence of harmonics, which are crucial for species-level identification. To supplement manual annotation, I also explored computational analysis using Python, particularly the librosa library for audio processing and spectrogram generation. This allowed for a deeper understanding of the temporal and spectral patterns in the calls.

Contents

Introduction	1
Objectives	2
Data Acquisition	3
Spectrogram Generation	6
Log Spectrogram v/s Linear Spectrogram	8
Filters	11
Manual Species Identifier	14
Discussion	15
Future Directions	16
References	19
Appendix	21
Code for Spectrogram Generation	21
Code for comparing Log Spectrogram and Linear Spectrogram	22
Code for Filters	23
Code for Manual Species Identification	27

Introduction

Understanding biodiversity starts with being able to tell species apart, a task that becomes surprisingly tricky when your subjects are tiny, fast, and nocturnal. Crickets, like many orthopteran insects, rely heavily on acoustic communication, especially for attracting a mate.[1] Each species has evolved distinct call structures, in rhythm, frequency, or pulse, that not only define their identity but also their ecological niches.

Weber et al. (1981) and their subsequent experiments provided compelling evidence that acoustic communication in field crickets, particularly *Gryllus campestris*, is species-specific and relies on temporal features of the male calling song. Using the Kramer treadmill, they tested female phonotactic responses to synthetic and natural variations of song structure. Their results showed that females track songs modulated at 30 Hz, the natural syllable repetition rate and discriminated against signals from closely related species such as *G. bimaculatus*, whose calls differ in temporal pattern and fine structure.[10]

These findings indicate that cricket calls encode species-specific cues, particularly in their syllable rate and chirp pattern, which are critical for conspecific recognition. The rejection of ‘wrong songs’ by females, even with subtle alterations in temporal structure, suggests that these acoustic parameters can serve as reliable features for species-level classification in both behavioral and computational systems. Later ecological studies began increasingly to rely on manual annotation of insect acoustic characteristics to assess species presence, behavior, and biodiversity. However, this process is labor intensive and prone to human error.[9] Thanks to advances in machine learning (ML) and deep learning (DL), researchers have begun automating species identification and quantifying the richness of habitat species directly from sound recordings. For example, Faiß & Stowell (2023) showed that deep learning frameworks, especially CNNs with advanced audio front-ends (like LEAF), can automatically detect and classify field-recorded insect sounds across dozens of Orthoptera and Cicadidae species, enabling robust biodiversity monitoring.[5] Moreover, recent reviews confirm that passive acoustic monitoring (PAM) combined with automated classification is transforming ecological

surveys—making them scalable, non-invasive, and capable of delivering species-level resolution and metrics such as community acoustic space use, which correlates strongly with insect richness.[9]

Objectives

For the past two months, my project has revolved around some fundamental questions like Can individual cricket species be accurately identified based on their acoustic calls, specifically, through analysis of spatial and temporal features on a linear spectrogram? And can a machine be trained to perform this task as reliably like a skilled field ecologist? It also involved determining the sort of call feature through which we can differentiate between the species.

To explore this, I worked with field recordings of calling crickets, generously provided by a current Ph.D. researcher from the Eco-Evo Lab at NISER. After careful inspection and segmentation, these recordings yielded distinct and identifiable call segments that covered multiple species. Some recordings contained overlapping calls, complicating the identification process. Each segment was manually annotated using Raven Pro, where I documented acoustic parameters such as start time, end time, low and high frequency, delta frequency, pulse rate, peak frequency, and average power density. These quantitative parameters were supplemented with qualitative notes, including the number of syllables per chirp and the presence of harmonic overtones, features that often hold the key to species-level resolution in acoustically cryptic taxa.

However, manual annotation was not scalable, especially when the goal was long-term monitoring across large landscapes. For that reason, it was essential to think to develop a classifier capable of identifying species-specific calls. To move in this direction, I turned to Python, learning the use of librosa, a powerful audio processing library. With librosa, I was able to generate linear and mel spectrograms, extract temporal and spectral features, and experiment with visualization tools. The idea here was not to build a perfect classifier in one go, but to start seeing how statistical patterns in frequency, time, and intensity relate to species identity. Even with basic features like

pulse rate and peak frequency, the models could already hint at separable clusters — a promising sign that more sophisticated approaches (like CNNs on spectrograms) could further enhance classification performance.

Data Acquisition

A total of 41 field recordings were obtained, each corresponding to a different species of cricket. Most of them were clean recordings, and we were able to clearly distinguish between noise and call. For each recording, our objective was to characterize the call by isolating a single representative chirp from the signal. Spectrograms were generated using short-time Fourier transforms to visualize the time-frequency structure of each recording in Raven Pro to confirm the presence and identity of species-specific call elements, as shown in **Figure 1**. We focused on energy-rich regions that were visually dark in color. These segments were the acoustic signal exhibited high intensity over time and frequency, and they it was also confirmed by listening to these high-intensity segments with the Raven Pro selection tool. To extract these features, one prominent chirp or syllable was manually chosen from each spectrogram. Each spectrograms corresponding to its recording was sampled for approximately 10 regions, regions which had our required chirp. Some recordings' spectrograms contained multiple harmonics or syllables, particularly in species with complex calls. Listening to the calls through Raven Pro helped verify the calls. Acoustic parameters such as start time, end time, low and high frequency, delta frequency, pulse rate, peak frequency, and average power density were extracted from the region using Raven Pro software. The software automatically extracted a set of acoustic features from the highlighted selection. These features were calculated based on the time-frequency representation of the sound and included parameters such as minimum and maximum frequency, bandwidth, peak frequency, duration, and energy. Raven Pro processed the selected time segment and applied Fast Fourier Transform (FFT) to analyze frequency components, then compiled the measurements into a data table. This table provided a structured summary of each selected signal, allowing users to export or analyze key sound characteristics across

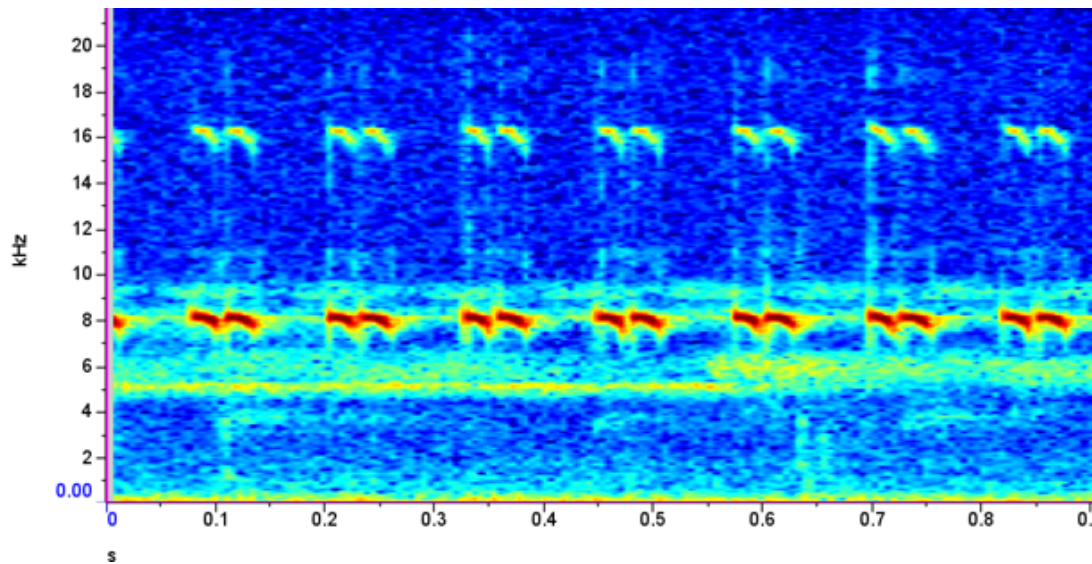
multiple selections. These quantitative parameters were supplemented with qualitative information, such as the presence of harmonic overtones. These features were chosen to capture the temporal and spectral characteristics of the calls, which are required for species-level classification and individual identification.[7] The data were then stored in a spreadsheet for all call recordings, as shown in **Table 1**. Pulse rate (Hz) was estimated for species with discrete, rhythmic chirps. However, for approximately 19 species, this could not be computed due to the continuous or highly overlapping nature of their calls, which lacked clearly separable pulses. We suspect that cleaner recordings might be necessary for reliably estimating pulse rates and distinguishing such call elements in these cases. To obtain a stable and representative feature vector for the individual, we then computed the mean of each feature across all the chirps in that region. This averaging process helped smooth out variability between individual chirps and capture the central tendencies of the individual's calling behavior. This standardized dataset formed the input for subsequent classification and clustering analysis aimed at evaluating inter-species differences and potential for automatic identification. Mathematically, the pulse rate P is defined as the number of discrete pulses N occurring over a time duration T , measured in seconds:

$$P = \frac{N}{T} \quad (1)$$

Where,

- P is the pulse rate in pulses per second (Hz),
- N is the total number of pulses or syllables observed in the call segment,
- T is the duration (in seconds) over which those pulses occur.

To ensure accurate estimation, pulses were counted manually from the spectrogram through visual-auditory confirmation using Raven Pro, especially for calls with clearly separated syllables. For recordings with continuous calling or overlapping syllables, where distinguishing individual pulses was not possible, the pulse rate could not be reliably calculated and was excluded from further analysis.



The x-axis represents time (seconds), while the y-axis shows frequency (kHz). The intensity of coloration reflects the energy at each frequency-time bin, with red and yellow indicating higher amplitude regions and blue indicating lower amplitude or background noise. Distinct harmonics can be observed, with dominant energy bands centered around 7-8 kHz and additional higher frequency components near 15-17 kHz. These periodic acoustic pulses represent individual chirps emitted by the calling male. The temporal regularity and multi-harmonic structure are characteristic features of this species' advertisement call.

Figure 1: Spectrogram of a cricket call segment.

Table 1: Golden cricket file

Sr. No	Begin Time (sec)	End Time (sec)	Low Freq (Hz)	High Freq (Hz)	Delta Freq (Hz)	Delta Time (sec)	Peak Freq (Hz)	Avg Power Density (dB FS/Hz)	Pulse Rate
1	0.072	0.141	7399.6	8544.1	1144.544	0.0684	8096.484	-40.02	29.239
2	0.198	0.268	7446.3	8544.1	1097.828	0.0694	8010.352	-39.68	28.818
3	0.322	0.392	7469.6	8590.8	1121.186	0.0704	8010.352	-39.84	28.409
4	0.443	0.513	7493.0	8544.1	1051.112	0.0704	8010.352	-39.91	28.409
5	0.571	0.640	7376.2	8544.1	1167.903	0.0694	8010.352	-40.07	28.818
6	0.693	0.763	7352.9	8544.1	1191.261	0.0704	8010.352	-40.18	28.409
7	0.815	0.884	7352.9	8544.1	1191.261	0.0684	8010.352	-40.27	29.239
8	0.942	1.011	7352.9	8614.2	1261.335	0.0684	8010.352	-40.47	29.239
9	1.064	1.134	7376.2	8590.8	1214.619	0.0699	8010.352	-40.25	28.612
10	1.187	1.256	7376.2	8544.1	1167.903	0.0689	8010.352	-40.12	29.027
		Average	7399.58	8560.45	1160.895	0.0694	8018.96	-40.081	28.822

Spectrogram Generation

Distinguishing between different species calls, especially those as complex as cricket songs, can be extremely challenging when relying solely on auditory perception. The human ear, while powerful, is not always equipped to detect subtle acoustic variations, especially in field recordings where background noise and overlapping signals can obscure details.[2] To overcome this, a visual approach is essential. This is where spectrograms become invaluable. A spectrogram transforms an audio signal into a two-dimensional visual representation, with time on the x-axis, frequency on the y-axis, and color intensity representing amplitude or power. This allows one to see how the energy of different frequencies changes over time, effectively turning sound into an image that can be analyzed.

I initially used Raven Pro, a software for manual acoustic analysis, and I quickly realized its limitations when it came to scalability and customization. Raven Pro is extremely helpful for quick visual inspections and labeling, but if I wanted to develop a species classifier or automate large-scale analyses, I needed a more flexible and programmatic approach.[4] This led me to Python, specifically the Librosa library. I learned how to generate and manipulate spectrograms through code, which allowed me to take full control over critical parameters like time window size, frequency resolution, and dB scaling. This was essential to tailor the spectrograms to the unique acoustic features of our data set. Generating spectrograms programmatically in Python is a necessary step toward building scalable and customizable analysis pipelines. When species-specific classifiers are developed or large volumes of bioacoustic data are processed, it becomes crucial to define spectrogram parameters, such as window size, frequency resolution, and dB scaling, according to the specific acoustic characteristics of the recordings. Coding also enables seamless integration of spectrogram generation into future workflows, including filtering, feature extraction, and machine learning tasks.

As part of this process, I learned to work with Librosa, a Python library prominently used for music and audio analysis.[8] It helps in loading audio files and processing them into waveforms and spectrograms. I used `librosa.load()` to import the **.wav** file and

extract the raw waveform and sampling rate. From there, I plotted the waveform using `librosa.display.waveshow()`, which visually showed the amplitude of the signal over time, as shown in **Figure 2**.

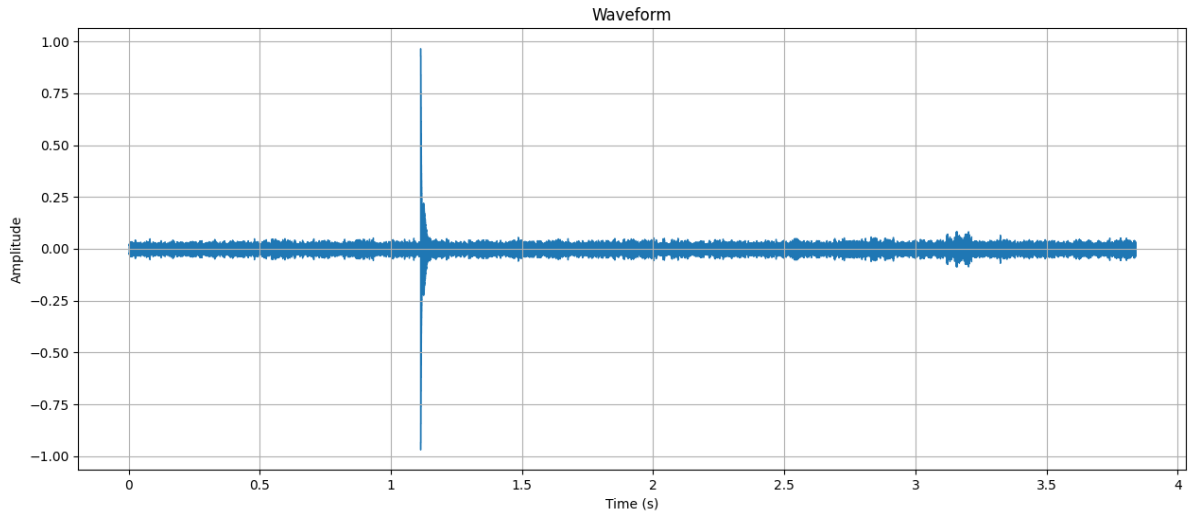


Figure 2: The waveform represents the amplitude of the sound signal over time. The sharp spike around 1 second indicates a high-amplitude sound event, likely a call or chirp.

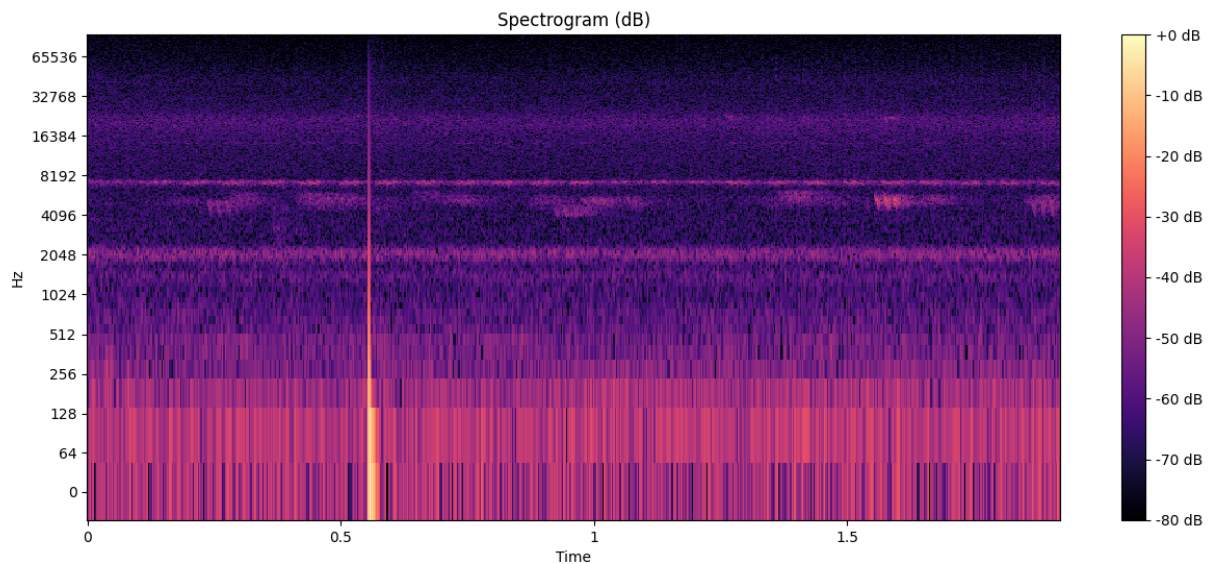


Figure 3: The spectrogram is a time-frequency representation of the sound, with color intensity representing sound power in decibels. Horizontal bands indicate dominant frequencies, and brighter regions highlight active acoustic energy, giving insight into the spectral richness and structure of the cricket call.

To generate the spectrogram, Short-Time Fourier Transform (STFT) was used via `librosa.stft()`, which breaks the audio into small overlapping chunks and calculates the frequency content in each slice. This gave a matrix of frequency bins over time, which was then converted to a decibel scale using `librosa.amplitude to db` making the energy differences easier to visualize. The resulting spectrogram was plotted using Matplotlib, a powerful visualization library, as shown in **Figure 3**. The frequency spectrum of cricket calls was visualized using `librosa.display.specshow()`, with `matplotlib.pyplot.colorbar()` providing a scale for interpreting energy intensity through color gradients. This made it possible to observe any patterns, dominant frequencies, and harmonics. Engaging in this analysis pipeline deepened my understanding of acoustic signals. It also provided me with foundational knowledge of time-frequency analysis, a crucial tool in bioacoustics and machine learning applications within ecological research.

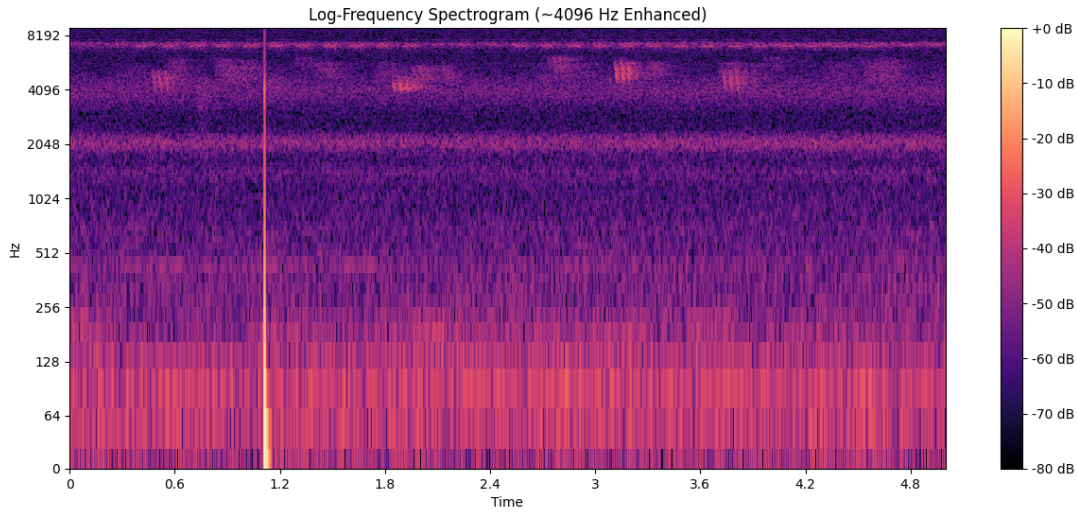
Log Spectrogram v/s Linear Spectrogram

Although the logarithmic spectrogram initially provided a broad overview of the frequency content, it resulted in the signal appearing compressed, particularly in the higher frequency ranges where biologically relevant components were seen in the spectrogram, because logarithmic scaling reduced the resolution of higher frequencies to emphasize lower frequency components more strongly. This compression made it difficult to visually separate the true signal from background noise and artifacts. To address this limitation and verify the fidelity of the extracted features, it became necessary to compare both logarithmic and linear spectrogram representations. The linear frequency spectrogram, with its uniformly spaced frequency bins, offered a clearer visualization of the energy distribution across the frequency spectrum. This comparison revealed that the linear spectrogram provided a more accurate and interpretable depiction of the signal structure, enabling better distinction between noise and the target acoustic features, thereby justifying its use for further analysis.

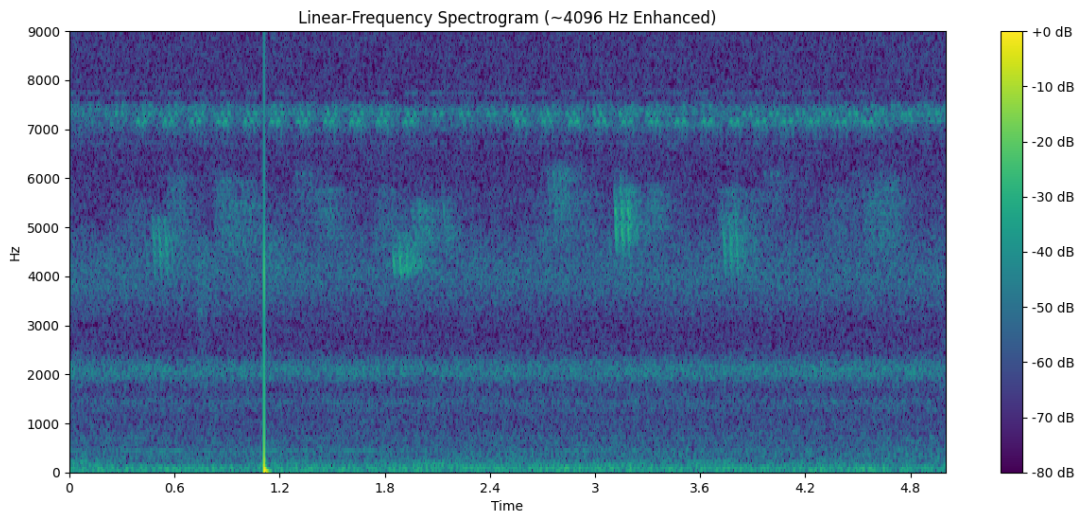
For this problem, we employed a Python-based signal processing code to enhance and analyze acoustic recordings using the `librosa` library. Initially, an audio file was

loaded and trimmed to the first five seconds to focus on a representative segment of the signal for downstream analysis. The Short-Time Fourier Transform (STFT) was calculated using a window size (`n_fft`) of 4096 and a hop length of 512 to break the audio signal into overlapping segments and analyze how its frequency content evolves over time. The window size determines the frequency resolution; larger values like 4096 provide finer detail in frequency but coarser resolution in time, making it suitable for identifying stable tonal components such as insect calls. The hop length controls the temporal spacing between windows, 512 provides good overlap, ensuring smooth transitions and minimizing data loss. We observed the change in the color at 4096 Hz similar to what we saw in the case of a call. To target biologically relevant frequency components, particularly those centered around 4096 Hz, we applied a custom frequency domain filtering approach. A gain mask was constructed to selectively amplify a target frequency band centered around 4096 Hz (± 1000 Hz), with gain weights that gradually decayed outward from the center frequency. A gain mask is essentially a filter or matrix applied to the spectral (frequency domain) representation of an audio signal, typically after computing the Short-Time Fourier Transform (STFT) to control the amplitude of specific frequency components. It functions like a frequency-specific volume control, allowing selective amplification based on defined criteria. In this case, frequencies within the target band were preserved or enhanced, while those outside were uniformly attenuated to 25% of their original amplitude. This selective filtering approach was designed to highlight components that likely correspond to the dominant calling frequency of the organism under study, while suppressing broadband noise and irrelevant spectral content. To evaluate the effect of this filtering, the processed signal was visualized using logarithmic and linear frequency spectrograms, as shown in **Figure 4**. The linear frequency spectrogram preserved the true spacing of the frequency bins, making it easier to distinguish the signal from background noise and assess filter performance. Finally, the modified spectral data were transformed back into the time domain using the inverse STFT (`istft`), and the resulting audio was saved as a new **.wav** file **cricket12.wav**, producing a cleaner, frequency-enhanced version of the orig-

inal recording. The audio was reconstructed to assess the impact of spectral filtering visually and audibly. Listening to the enhanced signal helps verify clarity and noise reduction, crucial for further analysis.



(a) Logarithmic frequency spectrogram displaying a wide dynamic range with emphasis on higher-frequency at 4096 Hz to 8192 Hz.



(b) Linear frequency spectrogram highlighting absolute frequency bands with uniform spacing. In both views, a Gaussian-shaped gain mask was applied to emphasize energy around the target band while moderately suppressing surrounding frequencies. Color bars indicate relative amplitude in decibels (dB).

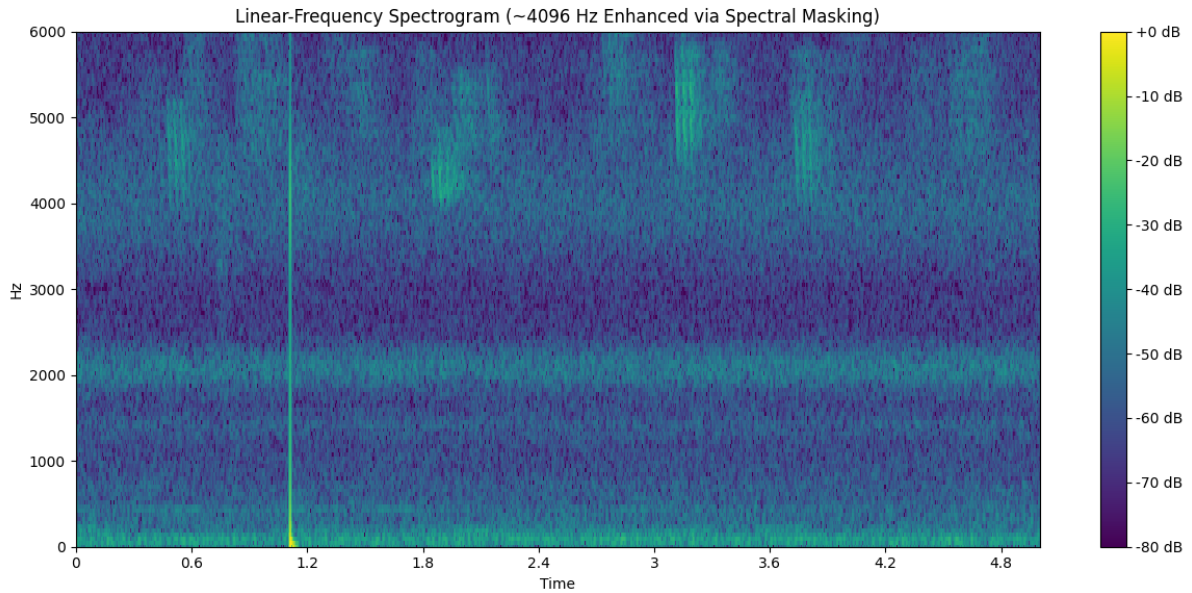
Figure 4: Spectrogram representations of a 5-second audio segment after frequency-specific enhancement around 4096 Hz.

Filters

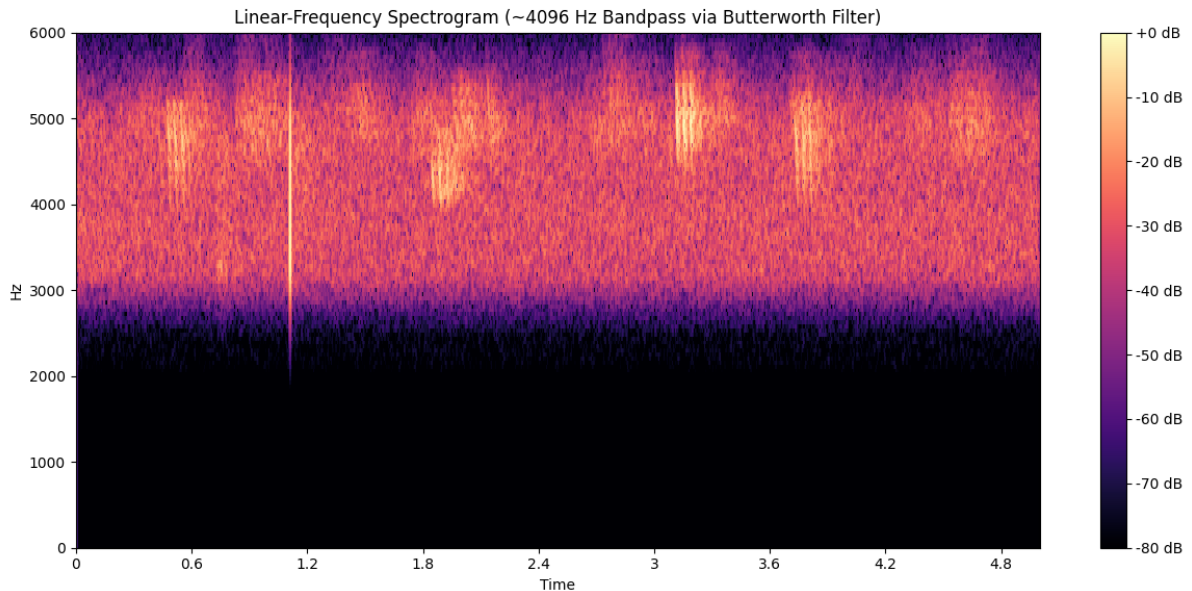
To evaluate the effectiveness of different filtering approaches in enhancing frequency components centered around 4096 Hz, a known peak frequency in this call recording, we applied two distinct methods to a 5-second audio segment. **(1)** a spectral masking technique implemented directly in the STFT domain, and **(2)** a true bandpass Butterworth filter applied in the time domain.[3] Both methods targeted a ± 1000 Hz band around the central frequency (4096 Hz). The resulting spectrograms were visually compared for clarity, noise suppression, and feature enhancement. Upon analysis, the Butterworth filter provided a more selective and cleaner enhancement of the call structure, reducing spectral leakage and improving contrast in the frequency band of interest. Thus, the Butterworth filter seemed to be more effective in isolating frequency-specific acoustic signatures in this context.

The primary objective of this code is to process acoustic recordings of cricket calls by enhancing and isolating biologically relevant frequency regions, specifically around 4096 Hz, to improve the clarity of chirps and analyze their temporal and spectral features. To achieve this, the code begins by loading a **.wav** file and extracting the first 5 seconds, which is a manageable chunk for analysis. Then it applies two different filtering techniques for comparison. The first method uses spectral masking, where frequencies near the target band (**4096 Hz $\pm$ 1000 Hz**) are selectively amplified while suppressing others using a soft attenuation approach. This does not zero out frequencies but reduces their contribution based on proximity to the center frequency. The second method applies a true Butterworth bandpass filter in the time domain, allowing only the desired frequency band to pass through, ensuring a more classical filtering operation. Spectrograms of both methods are plotted to visually compare their effectiveness in enhancing the call structure, as shown in **Figure 5**. After filtering, the signal processed by spectral masking is converted back to the time domain using inverse STFT and saved as a cleaned audio file. In addition to filtering, the code identifies high-energy call regions by setting a dB threshold and clustering adjacent time bins where the signal exceeds this threshold, which helps pinpoint individual chirps or call bursts.

These high-energy segments are then plotted on the spectrogram using bounding rectangles for visual reference. The code also mute all frequencies above 6200 Hz to remove high-frequency noise that may interfere with analysis. This allows for a cleaner inverse-STFT reconstruction of the call, which is again saved.



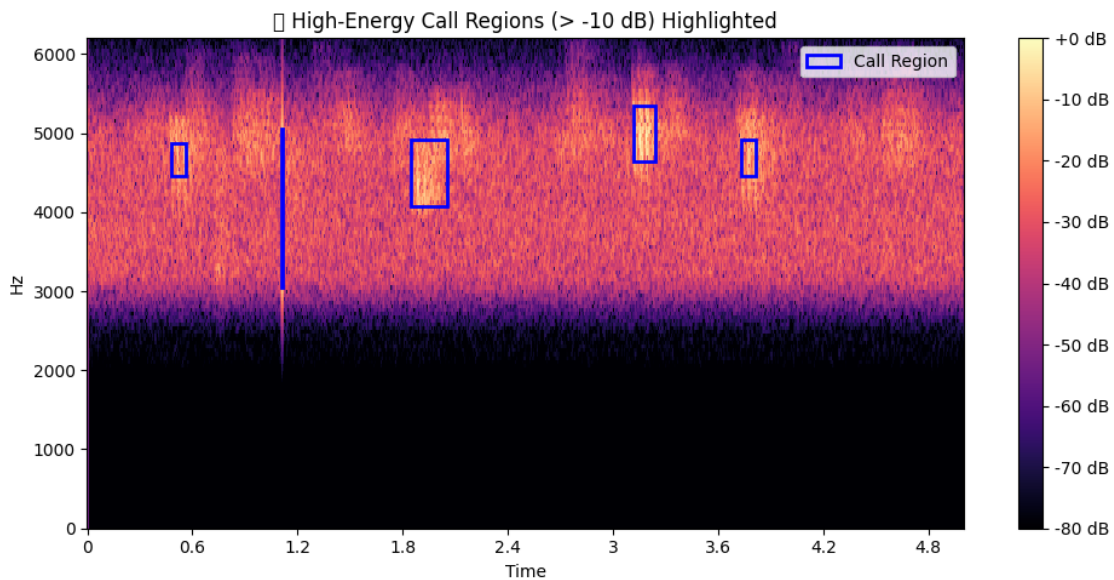
(a) shows the linear-frequency spectrogram of the call processed using a spectral masking method, where a gain mask was applied around the target frequency band (4096 ± 1000 Hz), reducing the amplitude of surrounding frequency bands by 75% to enhance signal contrast in the region of interest.



(b) shows the linear-frequency spectrogram of the same call processed with a 4th-order Butterworth bandpass filter targeting the same frequency range.

Figure 5: Comparison of signal enhancement techniques applied to a cricket call segment centered around 4096 Hz.

Overall, the code offers a powerful toolkit for isolating, visualizing, and extracting meaningful acoustic features from field recordings of insect communication. Once the code ran, it processed a 5-second chunk of the input **.wav** audio recording, specifically enhanced a frequency band around 4096 Hz using two methods: spectral masking and Butterworth bandpass filtering. It then generates and displays two spectrograms—one for each filtering method—so they can be visually compared. The Butterworth-filtered audio is further analyzed to detect high-energy regions (above -10 dB), which are assumed to correspond to individual call events or chirps. These call regions are identified, clustered based on temporal proximity, and highlighted as rectangular boxes on a spectrogram, as shown in **Figure 6**.



(a) The figure displays a linear-frequency spectrogram of a 5-second cricket call segment filtered using a Butterworth bandpass filter centered around 4096 Hz. Regions of interest were automatically detected by thresholding the amplitude in dB scale (threshold set at -10 dB), identifying areas of significant acoustic energy.

Figure 6: High-energy regions (> -10 dB) detected in the Butterworth-filtered spectrogram.

Each blue rectangle marks a segment where high-intensity sound energy exceeds the threshold, and these are interpreted as distinct chirp events produced by the calling insect. The algorithm clusters temporally adjacent frames with high amplitude to group them into discrete call events. This method not only aids in locating temporally resolved call bursts but also provides bounding frequency information for each detected

pulse. Such segmentation is useful for automated call detection, pulse counting, and temporal pattern analysis, especially in noisy environments where manual annotation is impractical. The clear separation of high-energy zones supports downstream analyses such as feature extraction, syllable structure characterization, and inter-call interval estimation.

Two filtered audio files are saved, one using spectral masking and another using Butterworth filtering, along with a third version where frequencies above 6200 Hz are muted. The output also includes a printed summary of the time, frequency range, and intensity of each detected call. These visualizations were created to compare the effectiveness of two filtering approaches in isolating call features from background noise. The Butterworth filter results in significantly more focused signal representation, suppressing broadband noise. In contrast, the spectral masking approach, while partially enhancing the desired band, retains more noise. This comparison illustrates that Butterworth filtering offers a cleaner, more interpretable spectro-temporal structure, especially suited for signal extraction and call segmentation in field recordings with moderate noise.

Manual Species Identifier

As the key acoustic features both the spectral and temporal domains of all 41 individual recordings was extracted using Raven Pro as mentioned earlier, we wanted to evaluate whether these six features alone could serve as a reliable fingerprint for species-level identification. To test this, we developed a simple matching system based on Euclidean distance, which quantifies how similar or different two feature vectors are in multidimensional space. The core idea was straightforward; if the features of an unknown or test call are numerically close to those of a known species, then it is likely to belong to that species. The code works by asking the user to manually enter six acoustic features of a test signal. The average values for the feature taken and were kept in an array. Then it computed the Euclidean distance between this input vector and each of the 43 stored reference vectors (each representing one distinct

call). The call whose reference vector has the minimum distance from the test vector is considered the closest match as shown in **Listing 1**. This also gives us a probable species identification and also a quantitative confidence measure in the form of the distance value.

```
1 Enter the 6 acoustic features of your test call:
2 1. Low Frequency (Hz): 9600
3 2. High Frequency (Hz): 22050
4 3. Delta Frequency (Hz): 12500
5 4. Delta Time (s): 0.39
6 5. Peak Frequency (Hz): 13800
7 6. Average Power Density (dB FS/Hz): -65
8
9 Closest match: S38
10 Euclidean Distance: 128.9730
```

Listing 1: MSI Output: Matching test call to closest known species

This code was developed as a proof of concept to explore whether just a small set of interpretable features could meaningfully differentiate calls. It serves as a lightweight and intuitive method, helping us to understand how informative these manually curated features are for classification. Additionally, such a feature matching approach can be used in the future to link field recordings to known species with minimal processing overhead.

Discussion

To begin analyzing species-specific features in field cricket recordings, we first isolated and visually examined high intensity acoustic segments, which were likely to contain biologically meaningful call elements. Using Raven Pro, we annotated 43 chirp segments across 41 recordings, extracting quantitative and qualitative features. However, manual annotation proved precise but unscalable for large-scale species classification. To address this, we shifted to a Python-based workflow using Librosa and Matplotlib to generate customized spectrograms suited to our dataset. This shift was necessary to build automated analysis pipelines, where defining window size, frequency resolution, and decibel scaling directly affects classifier performance and feature fidelity. A 5-second chunk of each recording was selected for standardized processing. We applied two different filtering techniques, spectral masking and Butterworth bandpass

filtering, targeting a ± 1000 Hz band around 4096 Hz, a dominant feature in our recording but can vary. Spectrograms generated after each filtering method revealed that Butterworth filtering yielded better contrast and cleaner call structures by effectively suppressing broadband and background noise, while spectral masking was less selective and retained more artifacts. The filtered signals were converted back into the time domain using inverse STFT and saved as new **.wav** files, forming cleaned versions of the original recordings. In addition, we muted frequencies above 6200 Hz to further reduce irrelevant high-frequency noise. To detect biologically relevant call regions, we implemented a method to identify high-energy acoustic segments by applying a decibel threshold on the filtered spectrogram. Temporally adjacent time bins above the threshold were clustered as individual chirp events and bounded with rectangles for visualization. This form of segmentation proved essential for accurately locating call parts, even in moderately noisy conditions.

Finally, to explore the classification potential of these extracted features, we developed a manual species identifier using the Euclidean distance. The system compared a user input feature vector with our dataset of known species recordings. The code returned the closest match and its corresponding distance score. Although not as sophisticated as deep learning approaches, this shows the probability that even a small set of interpretable features could meaningfully differentiate between calls. It also served as an intuitive tool to match new field recordings to reference data with minimal processing, paving the way for future integration into larger ML-based pipelines.

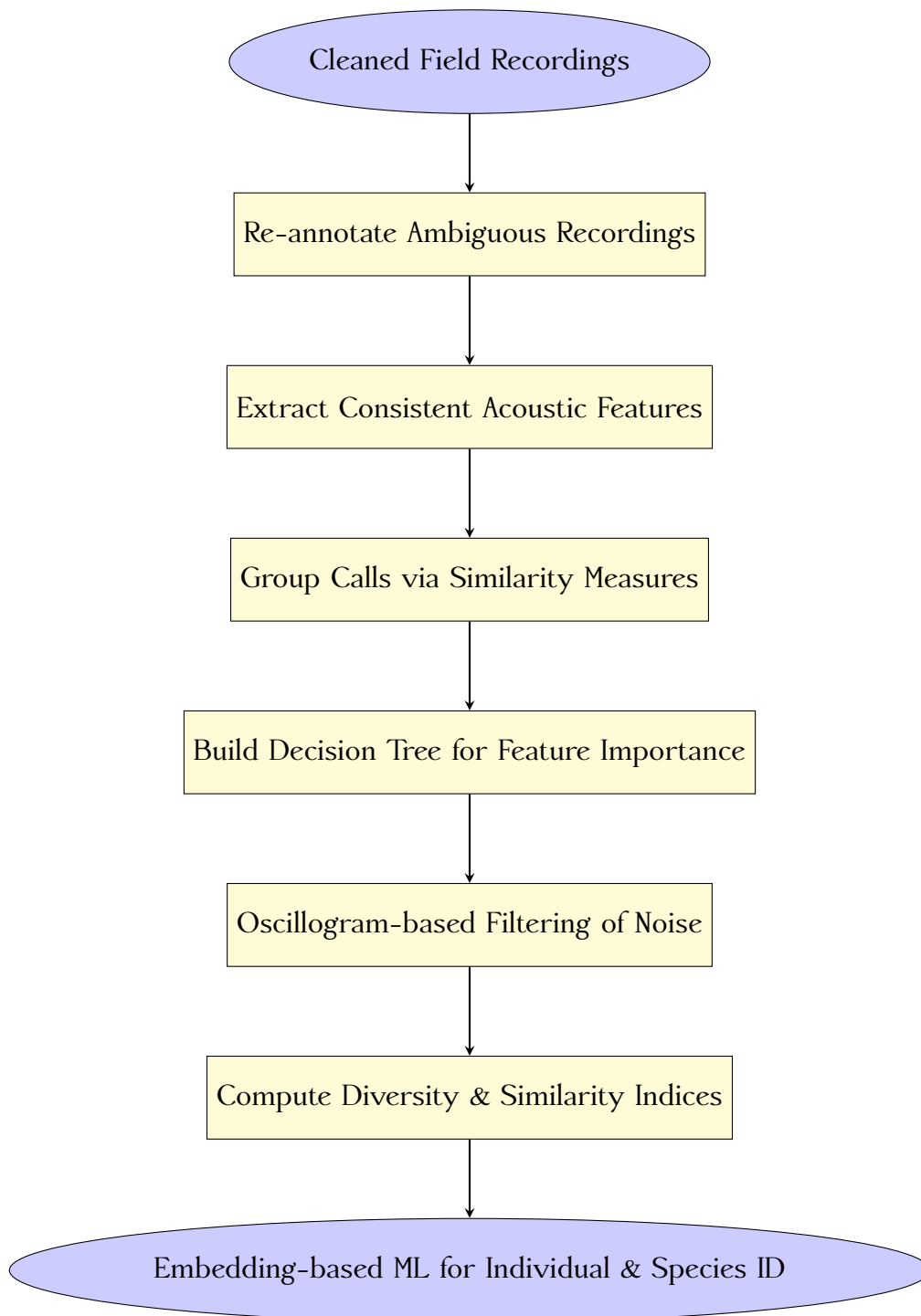
Future Directions

The future directions of this project will focus on expanding the current framework for more accurate and scalable species identification. A key component of this next phase involves reannotating the 19 recordings that currently exhibit ambiguous features. In several cases, we were unable to extract the pulse rate because the chirps were continuous without distinct pulses, while in others, excessive background noise obscured the signal, making reliable feature extraction difficult. These will be carefully

revisited to ensure higher quality feature extraction. From our existing data set of 41 field recordings, we have so far extracted 43 distinct call features. We plan to examine whether there are any recurring or overlapping feature sets among these recordings of the same species by comparing them using pairwise similarity measures. Any closely matching recordings will be grouped and considered as a single species unit, but only after multiple iterations of annotation and confirmation across recordings from different times and individuals to ensure consistency. Building on these confirmed features, we will design a decision tree to quantify the entropy and importance of each acoustic feature that contributed to the manual identification process. This will help us trace the decision-making pathway used during annotation, which can later be automated and applied directly to unprocessed recordings. For this, we plan to integrate oscillogram-based inspection to detect and isolate disturbance segments in the recordings by setting an amplitude threshold, ensuring only clean and informative portions of the signal are analyzed. This filtered data will form the basis for subsequent machine learning efforts that aim to mimic manual identification logic.

We will explore ecological indices such as species richness, Simpson’s index, Shannon’s diversity index, and eventually adopt more specialized acoustic diversity indices. These indices will provide a quantitative understanding of the structure and variability of calls within and across species, helping to assess interspecific acoustic differentiation. Generating these indices will require a more refined version of our dataset, for which consistent annotations across individuals will be crucial. Manual curation and labeling of cleaner call recordings from the same individuals is thus a high-priority step moving forward. We also plan to incorporate visualization tools, such as waveform plots, spectrograms, and temporal energy distributions, to visually validate our extraction of acoustic features. Furthermore, we will evaluate the feasibility of representation learning approaches as outlined in the work of Emanuel Kabuga, which use acoustic embeddings to capture high-level latent features to differentiate between individuals of the same species.[6] Adapting such deep learning-based methods will not only allow us to refine the clustering of similar calls, but may also enable individual identification

in populations. Together, these future directions are aimed at making the pipeline more robust, generalizable, and biologically meaningful, bridging computational precision with ecological insight. They will ultimately contribute toward building a scalable tool for species monitoring and behavioral analysis in natural habitats through bioacoustic signal processing.



References

- [1] Jeremy Anso, Amandine Gasc, Edouard Bourguet, Laure Desutter-Grandcolas, and Hervé Jourdan. Crickets as indicators of ecological succession in tropical systems, new caledonia. *Biotropica*, 54(5):1270–1284, 2022.
- [2] Elizabeth C Blake and Ian Cross. The acoustic and auditory contexts of human behavior. *Current Anthropology*, 56(1):81–103, 2015.
- [3] Stephen Butterworth. On the theory of filter amplifiers. *Wireless Engineer*, 7(6):536–541, 1930.
- [4] Cornell Lab of Ornithology. Raven pro: Interactive sound analysis software (version 1.6). <https://ravensoundsoftware.com/software/raven-pro/>, 2019. Computer software.
- [5] Marius Faiß and Dan Stowell. Adaptive representations of sound for automatic insect recognition. *PLOS Computational Biology*, 19(10):e1011541, 2023.
- [6] Emmanuel Kabuga, Diptarup Nandi, Stuart Burrell, Gciniwe Dlamini, Rohini Balakrishnan, Bubacarr Bah, and Ian Durbach. Acoustic individual identification in a species of field cricket using deep learning. *bioRxiv*, pages 2025–05, 2025.
- [7] Elly Knight, Tessa Rhinehart, Devin R de Zwaan, Matthew J Weldy, Mark Cartwright, Scott H Hawley, Jeffery L Larkin, Damon Lesmeister, Erin Bayne, and Justin Kitzes. Individual identification in acoustic recordings. *Trends in Ecology & Evolution*, 39(10):947–960, 2024.
- [8] Brian McFee, Colin Raffel, Daniel Liang, Daniel PW Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference*, pages 18–25. Citeseer, 2015.
- [9] Jérôme Sueur, Almo Farina, Amandine Gasc, Nadia Pieretti, and Sandrine Pavoine. Acoustic indices for biodiversity assessment and landscape investigation. *Acta Acustica united with Acustica*, 100(4):772–781, 2014.

- [10] John Thorson, Theo Weber, and Franz Huber. Auditory behavior of the cricket: li. simplicity of calling-song recognition in gryllus, and anomalous phonotaxis at abnormal carrier frequencies. *Journal of comparative physiology*, 146:361-378, 1982.

Appendix

Code for Spectrogram Generation

```
1 import librosa
2 import librosa.display
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # Load the audio file
7 filename = r'C:\Users\arman\OneDrive\Documents\Internship 2025\Renv12.
      wav'
8 y, sr = librosa.load(filename, sr=None)
9
10 # Plot the waveform
11 plt.figure(figsize=(12, 4))
12 librosa.display.waveshow(y, sr=sr)
13 plt.title("Waveform")
14 plt.xlabel("Time (s)")
15 plt.ylabel("Amplitude")
16 plt.grid(True)
17 plt.tight_layout()
18 plt.show()
19
20 # Compute the Short-Time Fourier Transform (STFT)
21 D = librosa.amplitude_to_db(
22     np.abs(librosa.stft(
23         y,
24         n_fft=2048,
25         hop_length=1024,
26         window='hann'
27     )), ref=np.max)
28
29 # Plot the spectrogram
30 plt.figure(figsize=(12, 4))
31 librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='log')
32 plt.colorbar(format='%+2.0f dB')
33 plt.title('Spectrogram (dB)')
34 plt.tight_layout()
35 plt.show()
```

Listing 2: Waveform and spectrogram visualization using Librosa

Code for comparing Log Spectrogram and Linear Spectrogram

```
1 import librosa
2 import librosa.display
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import soundfile as sf
6
7 # LOAD AUDIO
8 filename = "20241027_204500.WAV"
9 y, sr = librosa.load(filename, sr=None)
10
11 # TRIM TO FIRST 5 SEC CHUNK
12 window_duration = 5
13 samples_per_window = int(window_duration * sr)
14 y_chunk = y[:samples_per_window]
15
16 # STFT PARAMETERS
17 n_fft = 4096
18 hop_length = 512
19 D = librosa.stft(y_chunk, n_fft=n_fft, hop_length=hop_length, window='
    hann')
20 frequencies = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
21
22 # ENHANCE TARGET BAND, SUPPRESS OTHERS (Not zeroing)
23 target_freq = 4096
24 band_width = 1000
25 lower = target_freq - band_width
26 upper = target_freq + band_width
27
28 suppression_factor = 0.25 # 25% of original amplitude for suppressed
    bands
29
30 # Construct gain mask
31 gain_mask = np.full_like(D, suppression_factor, dtype=np.float32)
32 for i, freq in enumerate(frequencies):
33     if lower <= freq <= upper:
34         distance = abs(freq - target_freq)
35         weight = 1 - (distance / band_width) * (1 - suppression_factor)
36         gain_mask[i, :] = weight
37
38 # Apply the mask
39 D_filtered = D * gain_mask
40
41 # PLOT BOTH LOG AND LINEAR SPECTROGRAMS VERTICALLY
42 D_filtered_db = librosa.amplitude_to_db(np.abs(D_filtered), ref=np.max)
43
44 plt.figure(figsize=(10, 10))
45
46 # Subplot 1: Log-Frequency Spectrogram (Top)
47 plt.subplot(2, 1, 1)
48 librosa.display.specshow(D_filtered_db, sr=sr, hop_length=hop_length,
49                          x_axis='time', y_axis='log', cmap='magma')
50 plt.title("Log-Frequency Spectrogram (~4096 Hz Enhanced)")
51 plt.colorbar(format="%+2.0f dB")
52 plt.ylim(0, 9000)
53
54 # Subplot 2: Linear-Frequency Spectrogram (Bottom)
```

```

55 plt.subplot(2, 1, 2)
56 librosa.display.specshow(D_filtered_db, sr=sr, hop_length=hop_length,
57                           x_axis='time', y_axis='linear', cmap='viridis')
58 plt.title("Linear-Frequency Spectrogram (~4096 Hz Enhanced)")
59 plt.colorbar(format="%+2.0f dB")
60 plt.ylim(0, 9000)
61
62 plt.tight_layout()
63 plt.show()
64
65 # INVERSE STFT TO GET CLEANED AUDIO
66 y_filtered = librosa.istft(D_filtered, hop_length=hop_length)
67
68 # SAVE CLEANED AUDIO
69 sf.write("cricket12.wav", y_filtered, sr)
70 print(" Saved filtered 5-sec audio as 'cricket12.wav'.")

```

Listing 3: Frequency band enhancement and filtering around 4096 Hz

Code for Filters

```

1  import librosa
2  import librosa.display
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import soundfile as sf
6  from scipy.signal import butter, filtfilt
7  from sklearn.cluster import DBSCAN
8  import pandas as pd
9
10 # LOAD AUDIO
11 filename = "20241027_204500.WAV"
12 y, sr = librosa.load(filename, sr=None)
13
14 # TRIM TO FIRST 5 SEC CHUNK
15 window_duration = 5
16 samples_per_window = int(window_duration * sr)
17 y_chunk = y[:samples_per_window]
18
19 # STFT PARAMETERS
20 n_fft = 4096
21 hop_length = 512
22 D = librosa.stft(y_chunk, n_fft=n_fft, hop_length=hop_length, window='
    hann')
23 frequencies = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
24
25 # ENHANCE TARGET BAND, SUPPRESS OTHERS (Not zeroing)
26 target_freq = 4096
27 band_width = 1000
28 lower = target_freq - band_width
29 upper = target_freq + band_width
30
31 suppression_factor = 0.25 # 25% of original amplitude for suppressed
    bands
32
33 # Construct gain mask
34 gain_mask = np.full_like(D, suppression_factor, dtype=np.float32)

```

```

35 for i, freq in enumerate(frequencies):
36     if lower <= freq <= upper:
37         distance = abs(freq - target_freq)
38         weight = 1 - (distance / band_width) * (1 - suppression_factor)
39         gain_mask[i, :] = weight
40
41 # Apply the mask
42 D_filtered = D * gain_mask
43
44 # TIME-DOMAIN FILTER: TRUE BANDPASS BUTTERWORTH FILTER
45 def butter_bandpass(lowcut, highcut, sr, order=5):
46     nyquist = 0.5 * sr
47     low = lowcut / nyquist
48     high = highcut / nyquist
49     b, a = butter(order, [low, high], btype='band')
50     return b, a
51
52 lowcut = 4096 - 1000
53 highcut = 4096 + 1000
54 b, a = butter_bandpass(lowcut, highcut, sr, order=4)
55 y_butter_filtered = filtfilt(b, a, y_chunk)
56
57 # PLOT BOTH LINEAR SPECTROGRAMS VERTICALLY
58 plt.figure(figsize=(10, 10))
59
60 # Subplot 1: Linear Spectrogram (Spectral Masked)
61 plt.subplot(2, 1, 1)
62 D_filtered_db = librosa.amplitude_to_db(np.abs(D_filtered), ref=np.max)
63 librosa.display.specshow(D_filtered_db, sr=sr, hop_length=hop_length,
64                          x_axis='time', y_axis='linear', cmap='viridis')
65 plt.title("Linear-Frequency Spectrogram (~4096 Hz Enhanced via Spectral
66          Masking)")
67 plt.colorbar(format="%+2.0f dB")
68 plt.ylim(0, 6000)
69
70 # Subplot 2: Linear Spectrogram (True Butterworth Filtered)
71 plt.subplot(2, 1, 2)
72 D_butter = librosa.stft(y_butter_filtered, n_fft=n_fft, hop_length=
73                        hop_length)
74 D_butter_db = librosa.amplitude_to_db(np.abs(D_butter), ref=np.max)
75 librosa.display.specshow(D_butter_db, sr=sr, hop_length=hop_length,
76                          x_axis='time', y_axis='linear', cmap='magma')
77 plt.title("Linear-Frequency Spectrogram (~4096 Hz Bandpass via
78          Butterworth Filter)")
79 plt.colorbar(format="%+2.0f dB")
80 plt.ylim(0, 6000)
81
82 plt.tight_layout()
83 plt.show()
84
85 # INVERSE STFT TO GET CLEANED AUDIO FROM MASK METHOD
86 y_filtered = librosa.istft(D_filtered, hop_length=hop_length)
87
88 # SAVE CLEANED AUDIO FROM MASK METHOD
89 sf.write("cricket12_masked.wav", y_filtered, sr)
90 print("Saved filtered 5-sec audio (masked STFT) as 'cricket12_masked.wav")

```

```

    '.")
90
91 # SAVE CLEANED AUDIO FROM TRUE FILTER
92 sf.write("cricket12_butter.wav", y_butter_filtered, sr)
93 print("Saved filtered 5-sec audio (Butterworth) as 'cricket12_butter.wav'
    '.")
94
95
96
97 #Code in case you want to know the high intensity Pressure coordinate
    with respect to the Time and
98 #frequency
99 '''threshold_db = -10
100
101 times = librosa.frames_to_time(np.arange(D_butter.shape[1]), sr=sr,
    hop_length=hop_length)
102 freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
103
104 high_energy_indices = np.argwhere(D_butter_db > threshold_db)
105 high_points = [(times[t], freqs[f], D_butter_db[f, t]) for f, t in
    high_energy_indices]
106 high_points_sorted = sorted(high_points, key=lambda x: x[2], reverse=
    True)
107
108 print("\n Top 10 high dB points (time [s], freq [Hz], dB):")
109 for i, (t, f, db) in enumerate(high_points_sorted[:5]):
110     print(f"{i+1:>2}: Time = {t:.3f} s, Freq = {f:.1f} Hz, dB = {db:.1f}
    ")'''
111
112
113
114 # Identify regions where dB > threshold
115 threshold_db = -10
116 times = librosa.frames_to_time(np.arange(D_butter.shape[1]), sr=sr,
    hop_length=hop_length)
117 freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)
118
119 # Get boolean mask of where power is above threshold
120 high_energy_mask = D_butter_db > threshold_db
121 high_energy_indices = np.argwhere(high_energy_mask)
122
123 # Group by time bins
124 from collections import defaultdict
125
126 time_bin_groups = defaultdict(list)
127 for freq_bin, time_bin in high_energy_indices:
128     time_val = times[time_bin]
129     freq_val = freqs[freq_bin]
130     db_val = D_butter_db[freq_bin, time_bin]
131     time_bin_groups[time_bin].append((time_val, freq_val, db_val))
132
133 # Now cluster adjacent time bins (like call bursts)
134 grouped_calls = []
135 sorted_time_bins = sorted(time_bin_groups.keys())
136
137 current_group = [sorted_time_bins[0]]
138 for i in range(1, len(sorted_time_bins)):
139     if times[sorted_time_bins[i]] - times[sorted_time_bins[i-1]] < 0.25:

```

```

140         current_group.append(sorted_time_bins[i])
141     else:
142         grouped_calls.append(current_group)
143         current_group = [sorted_time_bins[i]]
144 if current_group:
145     grouped_calls.append(current_group)
146
147 # Print results
148 print(f"\n Found {len(grouped_calls)} distinct high-energy time regions
(> {threshold_db} dB):")
149 for i, group in enumerate(grouped_calls):
150     group_times = [times[t] for t in group]
151     group_freqs = [pt[1] for t in group for pt in time_bin_groups[t]]
152     group_dbs = [pt[2] for t in group for pt in time_bin_groups[t]]
153     print(f"  Call {i+1}: Time {min(group_times):.2f}s to {max(
group_times):.2f}s | "
154           f"Freq Range {min(group_freqs):.1f} H z  {max(group_freqs):.1f}
Hz | "
155           f"Peak dB = {max(group_dbs):.1f}")
156
157 call_regions = []
158
159 for i, group in enumerate(grouped_calls):
160     group_times = [times[t] for t in group]
161     group_freqs = [pt[1] for t in group for pt in time_bin_groups[t]]
162     group_dbs = [pt[2] for t in group for pt in time_bin_groups[t]]
163
164     call_info = {
165         "call_id": i + 1,
166         "start_time": min(group_times),
167         "end_time": max(group_times),
168         "min_freq": min(group_freqs),
169         "max_freq": max(group_freqs),
170         "peak_db": max(group_dbs)
171     }
172     call_regions.append(call_info)
173
174 # Optional: convert to DataFrame
175 call_df = pd.DataFrame(call_regions)
176
177 plt.figure(figsize=(10, 5))
178 librosa.display.specshow(D_butter_db, sr=sr, hop_length=hop_length,
179                           x_axis='time', y_axis='linear', cmap='magma')
180 plt.colorbar(format="%+2.0f dB")
181 plt.title(" High-Energy Call Regions (> -10 dB) Highlighted")
182 plt.ylim(0, 6200)
183
184 # Plot rectangles from cached data
185 for i, row in call_df.iterrows():
186     rect = plt.Rectangle((row["start_time"], row["min_freq"]),
187                           row["end_time"] - row["start_time"],
188                           row["max_freq"] - row["min_freq"],
189                           edgecolor='blue', facecolor='none',
190                           linewidth=2, linestyle='-', label='Call Region'
191
192         if i == 0 else "")
193     plt.gca().add_patch(rect)
194
195 # Legend

```



```

194 handles, labels = plt.gca().get_legend_handles_labels()
195 by_label = dict(zip(labels, handles))
196 plt.legend(by_label.values(), by_label.keys())
197
198 plt.tight_layout()
199 plt.show()
200
201 #MUTE ALL FREQUENCIES ABOVE 6200 Hz IN SPECTRAL DOMAIN
202 freq_limit_idx = np.where(frequencies > 6200)[0][0] # First index where
           freq > 6200
203 D_filtered[:freq_limit_idx, :] = D_filtered[:freq_limit_idx, :] # Keep
           below 6200
204 D_filtered[freq_limit_idx:, :] = 0 # Zero out above 6200 Hz
205
206 # INVERSE STFT TO GET CLEANED AUDIO FROM MASK METHOD
207 # (AFTER zeroing above 6200 Hz)
208 y_filtered = librosa.istft(D_filtered, hop_length=hop_length)
209
210 # SAVE CLEANED AUDIO FROM MASK METHOD (only up to 6200 Hz)
211 sf.write("cricket12_masked.wav", y_filtered, sr)
212 print("Saved filtered 5-sec audio (masked STFT,      6200 Hz) as '
           cricket12_masked.wav'.")

```

Listing 4: Comparison of spectral masking and Butterworth filtering centered at 4096 Hz for 5-second audio segment.

Code for Manual Species Identification

```

1 from scipy.spatial.distance import euclidean
2
3 # Array of the Species number and feature values
4 species_names = ["S1", "S2", "S2", "S3", "S4", "S5", "S6", "S7", "S8", "
           S9",
5                 "S10", "S11", "S12", "S13", "S14", "S15", "S16", "S17", "S18", "S19"
6                 ,
7                 "S19", "S20", "S20", "S21", "S22", "S23", "S24", "S24", "S25", "S25"
8                 ,
9                 "S26", "S26", "S27", "S28", "S29", "S30", "S31", "S31", "S32", "S32"
10                ,
11                "S33", "S34", "S34", "S35", "S36.1", "S36.2", "S37", "S37", "S38", "
           S39.1", "S39.2",
12                "S40.1", "S40.2", "S41"]
13
14 low_freqs = [7399.58, 6440.43, 13661.81, 7358.1971, 4789.6596,
           7175.4375, 4935.699, 8710.457, 4015.8569
15 , 4971.3145, 4203.4894, 4410.0807, 2857.9244, 8805.201, 6050.1144,
           7513.6767, 12898.5166, 4174.1789,
16 7446.4823, 8403.8837, 17623.301, 4759.5009, 9500.5508, 4634.7639,
           3988.3495, 4138.2911, 5805.8255, 12047.1215
17 , 5321.8369, 11304.076, 5651.3621, 12175.9668, 2386.966, 5394.7574,
           4631.2137, 4958.8181, 6402.162
18 , 14064.3242, 2062.2814, 6343.835, 9466.0315, 6729.8439, 15046.3686,
           10336.0175, 11771.071, 6080.5164
19 , 3146.444, 6745.8803, 9525.1084, 470.9708, 7469.933, 11545.9224,
           7660.4738, 5454.2181]

```

```

19 high_freqs = [8560.45, 9196.68, 17290.87, 15026.5281, 7422.2348,
20               10207.3125, 6517.586444, 20760.1059, 6472.2567, 7197.7625
21               , 5976.7768, 5288.0603, 3446.7181, 17523.4455, 6969.6251, 11108.3823,
22               22050, 6294.975, 10245.4019, 11735.9223
23               , 20388.3495, 5838.2173, 11390.286, 7667.7214, 5141.7477, 5893.0141,
24               7203.8835, 13862.1498, 6337.8734, 12181.746
25               , 7392.3552, 13911.4679, 4113.8593, 7421.3592, 6807.6699, 5692.9904,
26               8659.8965, 16533.243, 5373.3495, 10882.2815
27               , 11620.6133, 9945.4689, 17632.9561, 16599.7781, 13720.8805, 6947.0984,
28               4062.9832, 7866.0951, 22050,
29               2661.6989, 8075.6532, 12901.7473, 9504.3102, 6702.423]
30
31 delta_freqs = [1160.895, 2756.25, 3629.0625, 7668.3312, 2632.5751,
32                3031.875, 1581.887444, 12049.6487
33                , 2456.3998, 2226.44775, 1773.2875, 877.979, 588.7939, 8718.2445,
34                919.510, 3594.7057, 9151.4834, 2120.7961
35                , 2798.9196, 3332.039, 2765.0486, 1078.7163, 1889.7352, 3032.9575,
36                1153.3981, 1754.7233, 1398.058, 1815.0287, 1016.0366
37                , 877.67, 1740.993, 1735.5006, 1726.8932, 2026.6018, 2176.4562,
38                734.1725, 2257.7342, 2468.919, 3311.068
39                , 4538.4466, 2154.5817, 3215.625, 2586.5875, 6263.7607, 1949.8091,
40                866.582, 916.5393, 1120.2148, 12524.8916
41                , 2190.7282, 605.72, 1355.8252, 1843.8362, 1248.2048]
42
43 delta_times = [0.0694, 0.0373, 0.03527, 0.07627, 0.08012, 0.14512,
44                0.401477778, 0.01265, 0.10695, 0.21525
45                , 0.03302, 0.531, 0.28137, 0.14456, 0.72604, 0.08402, 0.42953, 1.06983,
46                0.08593, 0.06982, 0.06923, 0.04265
47                , 0.04024, 0.23395, 0.16776, 0.30474, 0.34004, 0.33367, 0.09694,
48                0.08743, 0.32466, 0.29365, 0.12441, 0.06982
49                , 0.25023, 0.08668, 0.05203, 0.04567, 0.34621, 0.34846, 0.17747,
50                0.38258, 0.38417, 2.00163, 0.14107, 0.34395
51                , 0.01674, 0.01493, 0.04553, 0.40613, 0.09005, 0.58083, 0.54591,
52                0.70929]
53
54 peak_freqs = [ 8018.965, 8010.3515, 14961.2696, 10585.7227, 6003.457,
55               8544.3747, 5933.593667, 13068.75,
56               5437.5, 6287.695, 5288.5547, 4823.438, 3092.1677, 12262.5,
57               6477.1876, 8363.4962, 13359.1993,
58               5211.0355, 9375, 9731.25, 19275, 5437.5, 10706.25, 5625, 4500,
59               4893.75, 6562.5, 13312.5,
60               5981.25, 11812.5, 6562.5, 13443.75, 3186.914, 6322.1482, 5159.3557,
61               5409.1405, 7708.8867,
62               15512.5195, 2618.4372, 7743.3397, 11025, 8380.7229, 16813.1248,
63               11688.2225, 12356.25,
64               6562.5, 3764.0041, 7484.9417, 13901.8359, 714.9019, 7829.4727,
65               12360.0587, 8182.617, 6210.1753]
66
67 avg_powers = [-40.081, -27.632, -55.997, -36.667, -51.021, -42.134,
68               -61.37333333, -54.104, -46.129, -58.7175, -34.941, -59.769, -48.465,
69               -56.834, -59.258, -44.282, -61.102, -46.335, -50.553, -41.539,
70               -67.215, -27.113, -62.886, -71.095, -53.862, -60.507, -41.587,
71               -65.476, -51.126, -76.436, -39.325, -63.67, -32.879, -49.258, -77.12,
72               -59.409, -38.512, -59.398, -57.079, -54.761, -55.773, -62.467,
73               -77.225, -37.782, -54.345, -53.338, -60.013, -78.97, -70.917,
74               -96.827, -82.267, -66.662, -66.992, -51.435]
75
76 def get_user_input():

```

```

49     print("Enter the 6 acoustic features of your test call:")
50     low = float(input("1. Low Frequency (Hz): "))
51     high = float(input("2. High Frequency (Hz): "))
52     delta = float(input("3. Delta Frequency (Hz): "))
53     duration = float(input("4. Delta Time (s): "))
54     peak = float(input("5. Peak Frequency (Hz): "))
55     power = float(input("6. Average Power Density (dB FS/Hz): "))
56     return [low, high, delta, duration, peak, power]
57
58 def match_species(user_vector):
59     min_dist = float('inf')
60     best_species = None
61
62     for i in range(len(species_names)):
63         reference_vector = [
64             low_freqs[i],
65             high_freqs[i],
66             delta_freqs[i],
67             delta_times[i],
68             peak_freqs[i],
69             avg_powers[i]
70         ]
71         dist = euclidean(user_vector, reference_vector)
72         if dist < min_dist:
73             min_dist = dist
74             best_species = species_names[i]
75
76     return best_species, min_dist
77
78 if __name__ == "__main__":
79     user_features = get_user_input()
80     species, distance = match_species(user_features)
81     print(f"\n Closest match: {species}")
82     print(f" Euclidean Distance: {distance:.4f}")

```

Listing 5: Species identification using Euclidean distance on 6 acoustic features